Desarrollo Web en Entorno Cliente

Interacción con el usuario

Índice

Esquema 3

Material de estudio 4

5.1. Introducción y objetivos 4

5.2. Data Binding 4

5.3. Event Binding 17

5.4. Comunicación entre componentes 25

5.5. Decorador @Input. Del padre al hijo 27

5.6. Decorador @Output. Del hijo al padre 30

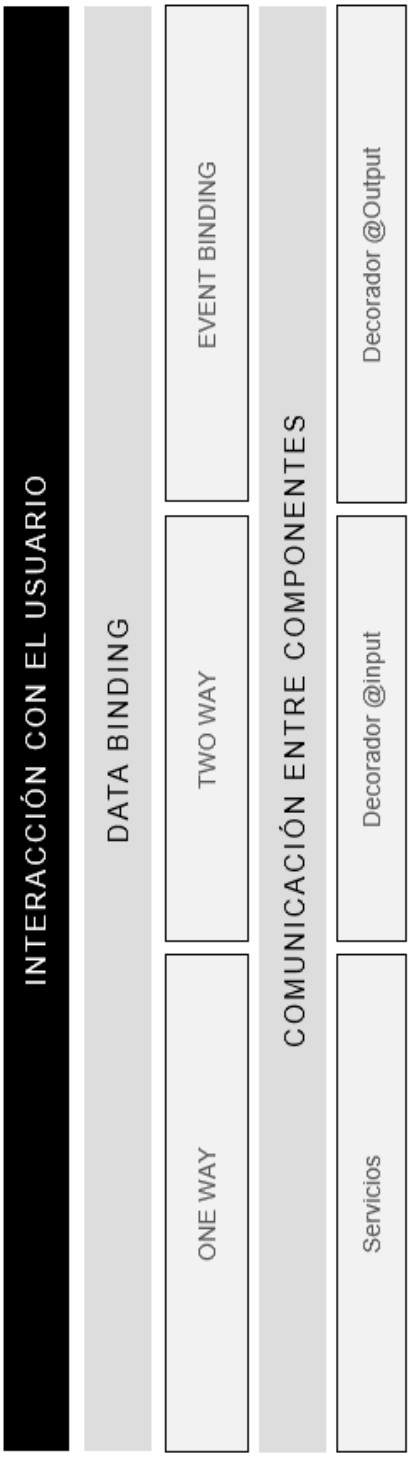
5.7. Referencias bibliográficas 33

A fondo 34

Entrenamientos 35

Test 38

Esquema


INTERACCIÓN CON EL USUARIO
DATA BINDING
ONE WAY
TWO WAY
EVENT BINDING
COMUNICACIÓN ENTRE COMPONENTES
Servicios
Decorador @input
Decorador @Output

Material de estudio

5.1. Introducción y objetivos

En este tema vamos a ver cómo podemos juntar la maquetación web clásica de html y css en Angular y así poder dar vistosidad a nuestros desarrollos, uniendo nuestra lógica de programación con nuestro HTML y dándole estilos a través de CSS.

Hasta el momento, las plantillas que hemos creado para nuestros componentes eran totalmente estáticas. Angular nos ofrece la posibilidad de ampliar nuestras plantillas e incorporar elementos dinámicos.

Durante el desarrollo de este tema se deben conseguir los siguientes objetivos:

5.2. Data Binding

Uno de los conceptos más importantes de este **framework** es cómo podemos hacer para enlazar los elementos creados en la clase de nuestro componente con la plantilla que finalmente renderizamos.

Vamos a ver las como comunicamos el archivo .ts con el .html**.**

Ya hemos visto que nuestro componente tiene un decorador que asigna, el selector con el que se carga el componente, el CSS que da los estilos al componente y el HTML que renderiza.

En este apartado vamos a analizar uno de los primeros elementos del Data Binding, este es uno de los conceptos más importantes en Angular para definir la

comunicación entre el componente y el DOM (archivo HTML). Este concepto hace muchos más fácil el pintar los datos variables dentro del HTML simplificando muchísimo el uso del DOM con respecto al uso de JavaScript nativo. El Data Binding son un conjunto de herramientas que Angular nos provee para estos menesteres.

De tal forma que no solo vamos a poder pintar datos en el HTML si no variar estilos y propiedades del DOM de una forma más sencilla. La comunicación dentro del componente se puede realizar en varios sentidos y dependiendo de este tenemos diferentes tipos de elementos.

Dentro del Data Binding hay tres tipos diferentes de comunicación entre plantilla, lógica y estilos. Veamos los tres modos y en los próximos apartados analizaremos en profundidad cada uno de ellos.

- One Way Data Binding (del componente al DOM): Interpolación, Property Binding y Class Binding.
- One Way Data Binding (del DOM al componente): Event Binding y $event.
- Two Way Data Biding (del componente al DOM y viceversa): FormsModule y [(ngModel)].

One Way Data binding mediante interpolación

Vamos a hablar de la interpolación como la forma más sencilla de visualizar propiedades de un componente en su template. Es la capacidad que tenemos de incluir datos dentro de nuestras plantillas.

Para realizar una interpolación solo necesitamos crear una propiedad dentro de la clase del componente. Este enlace se realiza incluyendo el código dinámico dentro de los símbolos {{ }}.

Es la estructura determinada por las llaves podemos incluir cualquier sentencia JavaScript válida excepto:

- Asignaciones.
- Creación de nuevas variables.
- Expresiones encadenadas.
- Incrementos/decrementos.

Cualquier sentencia que incluyamos dentro de las llaves será ejecutada como si de JavaScript se tratase.

Pero sin duda, el uso más habitual para esta técnica es la de mostrar el valor de las diferentes propiedades contenidas dentro de la clase del componente. El enlace entre la clase y la plantilla es directo y, por lo tanto, a través del nombre de la propiedad, podemos renderizar su valor en la plantilla.

Veamos un ejemplo:

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-primer-componente',
  templateUrl: './primer-componentes.component.html',
  styleUrls: ['./primer-componentes.component.css']
})
export class PrimerComponentesComponent implements OnInit {

  //creamos una propiedad de clase, es como la creación de una variable pero sin la palabra let.

  titulo: string;
  //inicializamos en el mismo punto de la creación
  numero: number = 2;
  constructor() {

    //las propiedades hay que inicializarlas y podemos hacerlo en el constructor o en la propia declaración de la propiedad. Para referirnos a la propiedad dentro de la clase usamos la palabra this.
    this.titulo = 'Titulo del Componente'
```

```
  }
  ngOnInit(): void {
  }

}
```

En el ejemplo de arriba vemos que hemos creado dos propiedades **título y numero**. Las propiedades se declaran por encima del constructor como si de dos variables globales se tratasen, pero sin usar la palabra let, var, const que usábamos en JavaScript.

Dichas propiedades aparte de declararla y asignarles un tipo, tenemos la obligación de inicializarlas, y esta tarea la podemos realizar en dos zonas, o en la misma declaración de la propiedad, asignándole un valor por defecto o en el constructor de la clase donde tendremos que usar la palabra reservada **this** para referirnos a la propia propiedad y asignarle un valor.

A través de esta propiedad y la interpolación podemos visualizar el valor de estos datos dentro del HTML de la siguiente forma.

```
<h1>{{ titulo }}</h1>
<p>{{ numero }}</p>
<p>Segundo</p>
```

De esta forma el resultado que aparecerá en nuestra pantalla será el siguiente.

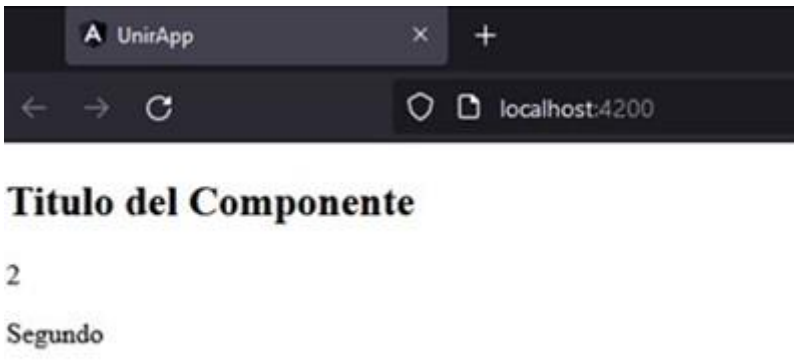


Figura 1. Renderizado de la interpolación de título y número. Fuente: Contenidos MFPFSDEV.

Aunque es la forma más sencilla de comunicar valores variables entre la lógica y el html, también podemos hacerlo a través de métodos dentro del componente.

Podemos declarar una función dentro del componente que devuelva un texto y dicha función pintarla como resultado dentro de nuestro HTML. Veamos su ejecución.

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-primer-componente',
  templateUrl: './primer-componentes.component.html',
  styleUrls: ['./primer-componentes.component.css']
})
export class PrimerComponentesComponent implements OnInit {

  //creamos una propiedad de clase, es como la creación de una variable pero sin la palabra let.

  titulo: string;
  //inicializamos en el mismo punto de la creación
  numero: number = 2;
  constructor() {
     //las propiedades hay que inicializarlas y podemos hacerlo en el constructor o en la propia declaración de la propiedad. Para referirnos a la propiedad dentro de la clase usamos la palabra this.
    this.titulo = 'Titulo del Componente'

  }

  ngOnInit(): void {
  }

  getParrafo(): string {
    return 'Este parrafo me esta llegando del return de una función dentro de mi componente.'
  }
```

```
}
```

Luego la pintamos en el HTML del componente de la siguiente forma:

```
<h1>{{ titulo }}</h1>
<p>{{ numero }}</p>
<p>{{ getParrafo() }}</p>
```

La función `getParrafo()` devuelve un texto que se renderiza en la interpolación del HTML dando como resultado lo siguiente.



Figura 2. Renderizado de un return de la función. Fuente: Contenidos MFPFSDEV.

Cuando el resultado que queremos pintar tiene etiqueta HTML la cosa cambia un poco por qué no podemos usar la interpolación como tal si no que debemos usar la propiedad innerHTML del DOM, aunque en este caso lo haremos de forma un poco distinta a como se hace en JavaScript.

Pongamos un ejemplo, vamos a crear una propiedad y un método que contengan un texto con etiquetas HTML en su interior de la siguiente forma.

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-primer-componente',
  templateUrl: './primer-componentes.component.html',
```

```
  styleUrls: ['./primer-componentes.component.css']
})
export class PrimerComponentesComponent implements OnInit {

  parrafo: string = 'En un lugar de la <strong>mancha</strong>'

  constructor() {}

  ngOnInit(): void {}

  getContenido(): string {
    return `Este contenido es un template literal y tiene un listado
            <ul>
              <li>opción 1</li>
              <li>opción 2</li>
            </ul>`
  }
}
```

La propiedad solo tiene una etiqueta strong y el método tiene un template literal que renderiza un listado no numerado.

Si usásemos una interpolación para pintar estos elementos de esta forma.

```
<p>{{ parrafo }}</p>
<p>{{ getContenido() }}</p>
```

El resultado obtenido no sería el esperado ya que las etiquetas se verían como si de texto se tratase, obteniendo este resultado.

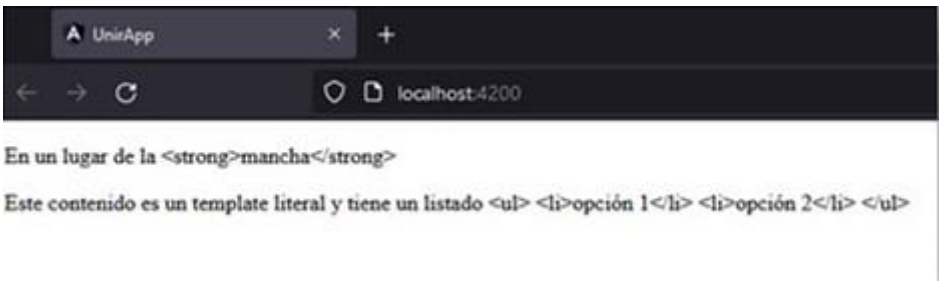


Figura 2. Renderizado erróneo de contenido HTML dentro de la plantilla. Fuente: Contenidos MFPFSDEV.

Como podéis ver las etiquetas forman parte del contenido y no son interpretadas por el navegador de forma correcta.

Pero ¿cómo podríamos visualizar el contenido de esos elementos dentro del HTML? Pues usando la propiedad de Angular [innerHTML] dentro de la etiqueta del HTML de la siguiente forma.

```
<p [innerHTML]="parrafo"></p>
<p [innerHTML]="getContenido()"></p>
```

Asignamos como atributo del párrafo dicha propiedad o igualamos al valor que retorna la función para así obtener ahora este resultado. Ahora si visualizándose el contenido HTML de forma correcta.

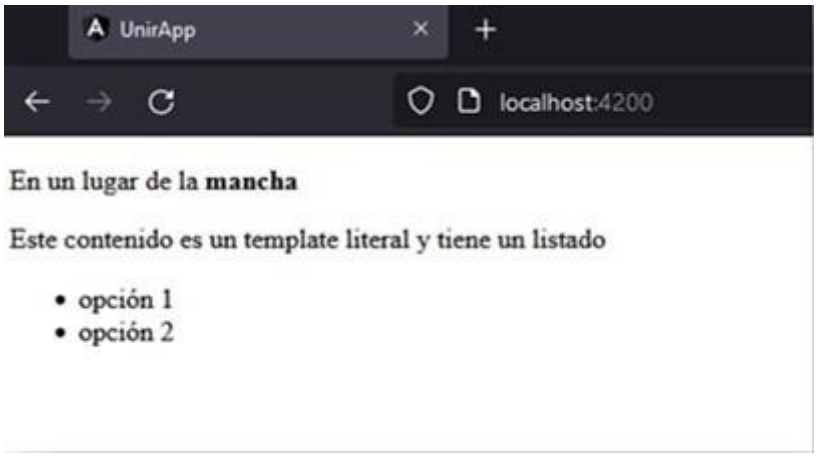


Figura 3. Renderizado correcto de contenido HTML dentro de la plantilla. Fuente: Contenidos MFPFSDEV.

Property Binding

Cuando necesitemos enlazar propiedades de componente a atributos del HTML debemos usar Property Binding de la misma forma que usamos la interpolación para meter valores dentro del contenido del HTML (que también podría usarse en estos casos).

La forma de implementarlo es la siguiente:

Le asignas los corchetes al atributo del DOM y al otro lado de la igualdad el valor de la propiedad del TS de la siguiente forma.

```
[atributoDOM] =”propiedadTs”
```

La asignación seguiría siendo igual que si utilizamos un literal, pero en este caso, el valor lo obtendremos de evaluar el contenido situado entre comillas.

Este contenido podría ser una de las propiedades de nuestra clase, la ejecución de un método o incluso una sentencia TypeScript.

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-primer-componente',
  templateUrl: './primer-componentes.component.html',
  styleUrls: ['./primer-componentes.component.css']
})
export class PrimerComponentesComponent implements OnInit {
  ruta: string = 'http://www.google.es'
  constructor() { }
  ngOnInit(): void {  }
}
```

En el ejemplo que tenemos arriba asignamos a una propiedad de clase ruta el valor de una url con destino a Google. Si queremos asignar dicho valor a una propiedad dentro del HTML, lo haremos de esta forma.

```
<a [href]="ruta">IR A GOOGLE</a>
```

De tal forma que cuando renderizamos el resultado ese enlace nos lleva de a Google de manera directa. Esto podría producirse con cualquier propiedad de HTML ya sea src de imagen, texto alternativo alt, id, class, title, en definitiva, cualquier propiedad del HTML puede tener asignada un valor variable en el componente.

Si eliminamos los corchetes, toma el contenido entre comillas como un literal. Con lo que el resultado esperado en el caso del ejemplo anterior es que pusiese dentro del `href="ruta"` en lugar de `href=http://google.es`

Podemos conseguir el mismo efecto con la siguiente sintaxis `href = {{ titulo }}`, aunque desde la documentación oficial de Angular, nos explican que debemos evitar esta forma de aplicarlo e intentar acostumbrarnos a la forma de corchetes ya que el resultado funciona mejor dentro del componente.

Class Binding

Las clases son propiedades también y pueden funcionar de manera muy similar a la de los ejemplos anteriores.

Es decir, podemos tener dentro del typescript una propiedad color que se la asigne un valor rojo que corresponda al valor de una clase de css de la siguiente manera.

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-primer-componente',
  templateUrl: './primer-componentes.component.html',
  styleUrls: ['./primer-componentes.component.css']
})
export class PrimerComponentesComponent implements OnInit {

  ruta: string = 'http://www.google.es'
  color: string = 'rojo'
  constructor() { }
  ngOnInit(): void { }

}
```

Al llevar esta propiedad a nuestro HTML usaremos los [] como forma de asignar el valor de la propiedad a la clase:

```
<a [href]="ruta" [class]="color">IR A GOOGLE</a>
```

Consiguiendo como resultado los siguiente:

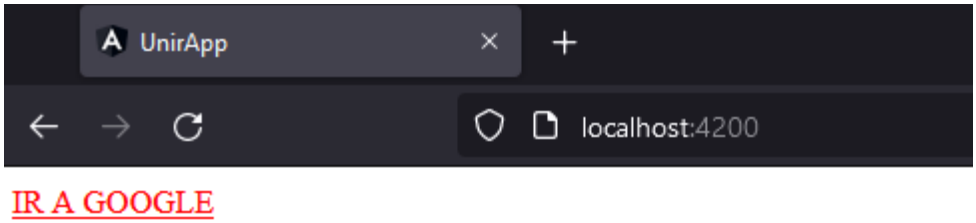


Figura 4. Resultado del Property Binding. Fuente: Contenidos MFPFSDEV.

De igual forma también nos permite especificar una clase de CSS para añadir a nuestro HTML, de tal forma que si le añadimos una propiedad del TS que tenga un valor booleano puedes activar el valor de una clase dando el valor true.

Veamos un ejemplo:

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-primer-componente',
  templateUrl: './primer-componentes.component.html',
  styleUrls: ['./primer-componentes.component.css']
})
export class PrimerComponentesComponent implements OnInit {

  ruta: string = 'http://www.google.es'
  color: string = 'rojo'
  stock: boolean = true;
  constructor() { }
```

```
  ngOnInit(): void { }

}
```

La propiedad stock es booleana y la vamos a usar para definir que nos pinte una clase rojo si su valor es true de la siguiente forma.

```
<a [href]="ruta" [class.rojo]="stock">IR A GOOGLE</a>

.rojo{
  color:red
}
```

Two way date biding: FormsModule y [(ngModel)]. Doble comunicación

Se trata de una de las técnicas más usadas para la recopilación de datos por parte del usuario.

Esta herramienta nos da la posibilidad de enlazar un campo de texto definido dentro de nuestra plantilla con cualquiera de las propiedades del componente, de una manera bidireccional.

Para poder hacer uso de esta característica utilizamos la directiva ngModel. Para poder hacer uso de esta directiva, dentro de la definición de nuestro módulo, debemos importar la librería FormsModule.

Nuestro app.module.ts quería de la siguiente forma.

```
import { BrowserModule } from '@angular/platform-browser';
import { NgModule } from '@angular/core';
import { FormsModule } from '@angular/forms'
...
@NgModule({
  ...
  imports: [
```

```
    BrowserModule, FormsModule
  ],
  ...
})
export class AppModule { }
```

Posteriormente podemos usar la directiva dentro de nuestras plantillas de la siguiente manera:

```
<input [(ngModel)]="textoInput" />
<h2>{{ textoInput }}</h2>
```

Lo que hacemos con la directiva [(ngModel)] es asociar una propiedad de Typescript con la directiva ngModel haciendo que así cualquier modificación en el input se guarde en la propiedad y viceversa, obteniendo así una comunicación bidireccional.

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-primer-componente',
  templateUrl: './primer-componentes.component.html',
  styleUrls: ['./primer-componentes.component.css']
})
export class PrimerComponentesComponent implements OnInit {

  textoInput: string = "Hola con la variable iniciada"
  constructor() { }

  ngOnInit(): void { }

}
```

En este caso concreto, el valor del campo de texto queda enlazado en ambos sentidos con la propiedad textoInput definida dentro de nuestra clase.

Si modificamos la variable, se modifica el valor del campo de texto y viceversa.

Esto es una forma alternativa a los formularios de angular que nos permite recoger datos por parte del usuario de nuestra aplicación asociando a una propiedad del TypeScript.

En el tema siguiente veremos como a través de eventos podemos usar el Two Way Data Biding para asociar por ejemplo a un objeto el resultado de dos inputs de tal forma que se recoja en cada una de las propiedades del propio objeto los valores del usuario introduce dentro de cada input.

5.3. Event Binding

El objetivo que buscamos en este apartado es interactuar con el usuario de nuestra web. Una página web no es un ente estático si no es un lugar donde el usuario puede interactuar y modificar elementos de la web, ya sea a través de formularios, botones o filtros, el usuario es una parte muy importante del desarrollo de nuestras aplicaciones ya que a través de su interacción recibimos muchísima información que nos ayuda a tomar decisiones.

Esto da lugar a los eventos, y estos están presentes en muchas aplicaciones y de diferentes formas. En JavaScript los manejábamos a través de listeners y Angular nos aporta una forma muy sencilla de manejarlos y recoger los valores que emite nuestra interfaz.

En este apartado vamos a ver las diferentes formas que tenemos de capturar los eventos y de mandar información, esta vez, desde el HTML hacia nuestro fichero TypeScript.

Creación de eventos en Angular

Event Binding nos permite implementar dentro de Angular un sistema de eventos o funciones que nos posibilita conectar el HTML y la interacción del usuario y cambiar el resultado.

El método que llamamos tras la ejecución del evento por parte del usuario nos permite recoger datos del propio DOM o cualquier dato que le pasemos por parámetros a dicha función. Para llamar a un evento invocamos el tipo de evento sin la palabra «on» entre paréntesis seguido de un igual y la llamada a la función definida dentro de Typescript.

Podemos utilizar los eventos definidos de manera nativa dentro del navegador o crear los nuestros propios.

Para poder definir qué evento estamos capturando utilizamos los paréntesis para limitarlos.

Vemos un ejemplo para capturar la acción de **click** sobre un botón.

```
<button (click)="hacerClick()">Pulsame</button>
```

Esta captura de código que visualizas es un elemento que se situación en el HTML del componente. Entre paréntesis puedes ver el tipo de evento sin la palabra «on» y entre las “” puedes ver el método de clase que se ejecuta cuando se produce dicho evento.

Este método tiene que estar definido dentro de la clase del componente de la siguiente forma.

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-primer-componente',
  templateUrl: './primer-componentes.component.html',
```

```
  styleUrls: ['./primer-componentes.component.css']
})
export class PrimerComponentesComponent implements OnInit {

  constructor() { }

  ngOnInit(): void { }

  hacerClick() {
    alert('has hecho click en el boton')
  }

}
```

Si ejecutas la interfaz lo que vas a poder observar si haces click en el botón es que esta interactúa y te devuelve una respuesta en forma de alerta.

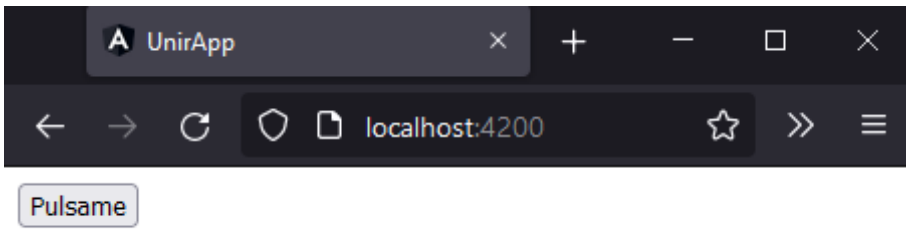


Figura 5. Resultado de la vista de componente antes de hacer click. Fuente: Contenidos MFPFSDEV.

Al pulsar en el botón la interfaz captura el evento llama al método de la clase y se produce el resultado, en este caso una alerta de JavaScript que nos lanza un mensaje de la siguiente forma.



Figura 6. Salida por pantalla de la resolución del evento. Fuente: Contenidos MFPFSDEV.

Si unimos la consecución de un evento con lo explicado en el tema anterior podemos asociar el resultado de varios inputs a través del Two Way Data Biding. Os recuerdo los pasos para poder realizar esta técnica:

Importar **FormsModule** en App-module.
Crear una propiedad dentro de la clase, esta vez será un objeto JSON.
Asociar a cada uno de los inputs una propiedad del objeto.
Cuando hagamos **click** en el botón sacaremos por consola el resultado del objeto ya relleno.

Como ya vimos cómo se hacían las importaciones en el tema anterior me voy a centrar solo en la creación y asociación de las propiedades del objeto y en la ejecución del evento.

```
export class PrimerComponentesComponent implements OnInit {

  //creamos la propiedad
  alumno: any;

  constructor() {
    //inicializamos los valores del objeto
    this.alumno = {
      nombre: "",
```

```
      edad: 0
    }
   }
}
```

En el fragmento de código anterior vemos como se declara un objeto alumno y lo inicializamos dentro del constructor con las propiedades nombre y edad, con los valores por defecto

En el siguiente fragmento de código vamos a ver como montamos el evento dentro de un botón y como asignamos a cada input los valores de los diferentes elementos.

```
<input type="text" [(ngModel)]="alumno.nombre" placeholder="introduce un nombre">
<input type="number" [(ngModel)]="alumno.edad" placeholder="introduce una edad">
<button (click)="capturarDatos()">Pulsame</button>
```

En el Typescript habrá que dar de alta el método de capturarDatos() donde simplemente mostraremos los valores del objeto alumno por consola.

```
export class PrimerComponentesComponent implements OnInit {
  //creamos la propiedad
  alumno: any;
  constructor() {
    //inicializamos los valores del objeto
    this.alumno = {
      nombre: "",
      edad: 0
    }
  }
  ngOnInit(): void { }

  capturarDatos() {
    console.log(this.alumno);
  }
}
```

El resultado que se produce en la consola del navegador es el siguiente.

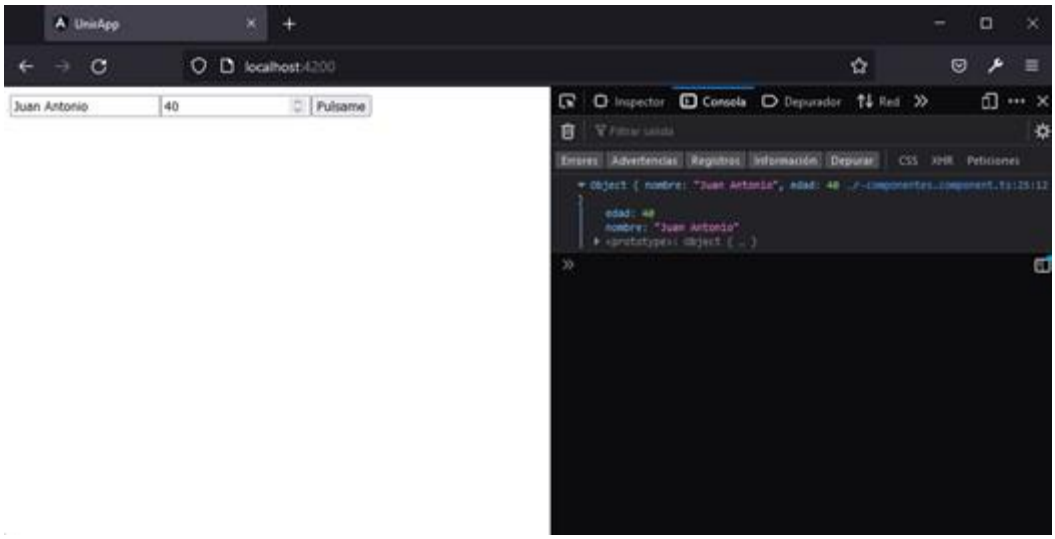


Figura 7. Captura de resultado del evento en consola al hacer click. Fuente: Contenidos MFPFSDEV.

Como puedes ver el tomar datos de esta forma combinando las dos técnicas me permite una comunicación bidireccional entre HTML y TypeScript. De esta forma podemos empezar a interactuar entre el usuario y nuestra aplicación.

Por descontado que no hace falta el crear objetos para asignar campos, pero es una buena práctica ya que en un único objeto tengo capturados todos los valores de los campos.

Los eventos que se pueden producir en una aplicación no son solo de click, por supuesto, sino que hay un abanico amplio de evento que pueden resultar interesantes y que se aplican de la misma forma

```
<div (mouseover)=handleOver()”>
<input type=”text” (keydown)=handleKey()”>
<form (submit)=handleSubmit()”>
```

$event

Dentro de los paréntesis del método que lanza el evento podemos insertar cualquier tipo de parámetro, ya sea un string, un número o un objeto completo. Esto podrá cambiar dependiendo de las necesidades que tengamos dentro de nuestro proyecto.

Uno de los argumentos más importantes que podemos pasarle a un evento es $event, ya que me permite enviar información del objeto que está lanzando en evento y poder capturar de él, cierta información necesaria para la ejecución de mis acciones. Depende del evento que estemos capturando podremos recuperar unas propiedades u otras de $event**.**

Volvamos al ejemplo del botón simple y veamos qué posibilidades nos ofrece en este caso pasarle como parámetro al método el $event**.**

```
<button (click)="capturarDatos($event)">Pulsame</button>
```

Como podéis observar en este caso, dentro de los paréntesis de la función tenemos como argumento de nuestra función el $event cuando se ejecute la acción descrita, en el método capturarDatos() tendremos toda la información que se produce del evento, como por ejemplo la posición en el escenario, el tipo de evento de click, y sobre todo y lo más importante el objeto que lanza el evento, en este caso un button de html.

```
capturarDatos($event:any) {
  console.log($event);
}
```

El resultado que nos arroja la consola de nuestro navegador es el siguiente (ver Fig.8). Como veis la información enviada es bastante extensa, y entre toda esa información el elemento más importante es el target ya que representa el elemento del DOM que tocamos.

El target al igual que en JavaScript nativo representa el elemento del DOM que ejecuta el evento, y a partir de este elemento podemos captura una propiedad del objeto, como por ejemplo el id, o el texto del botón con innerText, al igual que hacíamos en JavaScript. Por ejemplo $event en caso de un evento de teclado me

ofrece la posibilidad de sacar el código de tecla para diferenciar entre que tecla del teclado estoy haciendo keydown, a través del $event.keyCode.

Se debe recordar que a través de un evento podemos cambiar el resultado de cualquier propiedad de nuestro TypeScript lo que me permitiría cambiar una clase o un estilo a través de Property Biding o un texto a través de la interpolación.

Combinando todo lo aprendido hasta ahora, podemos empezar a desarrollar ya aplicaciones un poco más complejas con interacción del usuario.

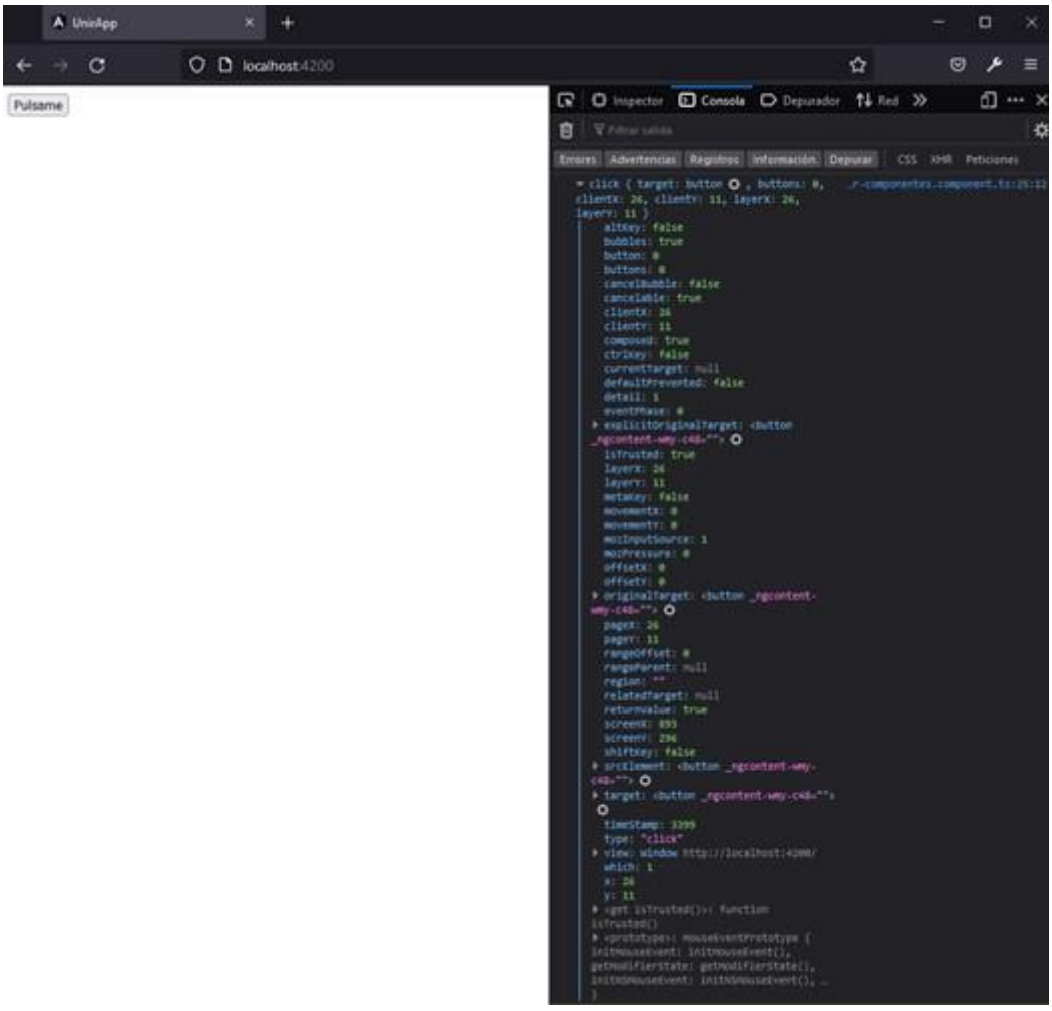


Figura 8. Captura de la salida de consola del navegador. Fuente: Contenidos MFPFSDEV.

5.4. Comunicación entre componentes

Dentro de una página web nos podemos encontrar diferentes tipos de componentes que realizan acciones distintas pero que tienen relación con cada uno de los otros componentes que les rodean, en mucho caso estos componentes deben comunicarse algunas cosas y para ello podemos usar diferentes formas.

Como hemos explicado en los objetivos del tema existen muchas formas de comunicar componentes, dependiendo si estos están relacionados entre sí o no.

Durante este curso iremos viendo formas de pasar información entre dos componentes, entre un componente padre y un componente hijo, entre el componente y la BBDD a través del servicio. Es decir que gran parte del acceso a los datos que tienen los componentes la obtienen de la comunicación que puede existir entre los diferentes métodos de comunicarse.

Por matizar un poco más esto nos podemos encontrar con los siguientes casos.

Un componente quiere pasarle información a otro componente que este contenido en él. Es decir, un componente padre quiere comunicar algo a su hijo. Es caso es bastante común dentro de la comunicación entre componentes. En este caso realizaremos la comunicación mediante un @Input.



Figura 9. Explicación gráfica de un input. Fuente: Contenidos MFPFSDEV.

Un componente que este contenido dentro de otro quiere comunicarle algo a su contenedor. Es decir, un componente **hijo** quiere comunicar algo a un componente **padre**. En este caso la comunicación se debe realizar a través de un evento personalizado o @Output.



Figura 10. Explicación gráfica del output. Fuente: Contenidos MFPFSDEV.

Dos componentes que no están relacionados entre sí quieren transmitirse información. En este caso al no haber ningún tipo de relación, directa, entre ellos hay que encontrar un tercer elemento que me permita comunicarme parcialmente con cada uno de los componentes, para ello vamos a usar diferentes técnicas

dependiendo de donde se encuentre los datos. Aunque la más común y que veremos más adelante en el curso es la comunicación usando un servicio.

Más adelante el servicio también nos aportará como funcionalidad la posibilidad de hacer consultas a la BBDD externa a la aplicación y poder traer datos para luego enviárselos a los otros componentes de la aplicación. Sería como el director de juego o base en baloncesto, que reparte el juego entre el resto de sus compañeros.

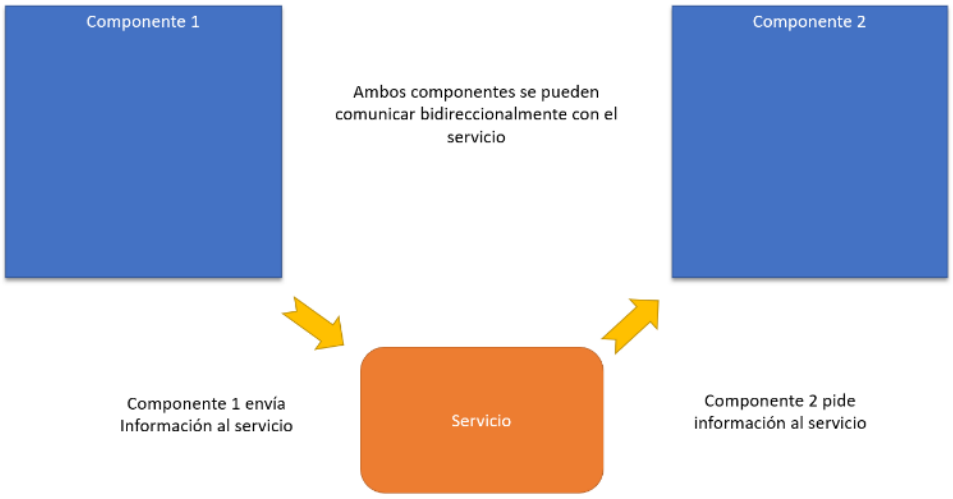


Figura 11. Comunicación a través del servicio. Fuente: Contenidos MFPFSDEV.

En este tema nos vamos a centrar exclusivamente en los dos primeros, y más adelante le daremos al servicio un estudio en profundidad donde veremos todas sus particularidades.

5.5. Decorador @Input. Del padre al hijo

Un decorador @input, como ya hemos explicado antes, nos permite comunicar un padre con su hijo y que este le pueda enviar diferentes valores al hijo para cambiar su estado, o simplemente pintar un resultado diferente.

Lo primero que tenemos que hacer para usar el Input es importarlo en el componente que vayamos a usarlo de la siguiente forma.

```
import { Component, Input } from '@angular/core';
```

El siguiente paso es decorar la propiedad con la que vamos a trabajar, para ello como con cualquier decorador le ponemos la función decoradora por delante de la propiedad la cual queremos convertir en Input. Es importante y TypeScript nos avisará de ello, que le pongamos el tipo del dato que va a almacenar ya sea un número o un string, etc. También si TypeScript está en modo estricto nos obligará a inicializar el valor. Esto como cualquier propiedad podréis hacerlo en la misma línea de la declaración o dentro del método constructor de la clase.

```
import { Component, OnInit, Input } from '@angular/core';

@Component({
  selector: 'app-primer-componente',
  templateUrl: './primer-componentes.component.html',
  styleUrls: ['./primer-componentes.component.css']
})
export class PrimerComponentesComponent implements OnInit {

 //Creación de un input e inicializamos el valor.
  @Input() texto: string = "";

  constructor() {
  }

  ngOnInit(): void { }

}
```

Ahora tenemos que llenar esa propiedad con un dato, y este se lo vamos a pasar a través del selector que carga el componente dentro de otro, ya sea el principal como otro cualquiera.

```
<section>
   <app-primer-componente [texto]="'Una cadena de texto que viene del padre'"></app-primer-componente>
</section>
```

Como podéis ver al crear una propiedad texto dentro de la etiqueta del componente, está la ponemos entre corchetes, para que admita el uso de sentencias JavaScript validas como valor del input, en este caso le hemos pasado una cadena de texto, de ahí que el texto este entre comillas simples.
Tal y como veíamos anteriormente, este tipo de atributos también pueden recoger su valor evaluando el contenido entre comillas, mediante la nomenclatura de corchetes.

Podemos definir un alias dentro de la llamada al decorador para especificar un nombre diferente para el atributo que usaremos en la declaración del tag

```
  //Creacion de un input
  @Input('miTexto') texto: string = "";
```

Si usamos ese alias tenemos que cambiar el valor del input en la etiqueta de creación del componente.

```
<section>
   <app-primer-componente [miTexto]="'Una cadena de texto que viene del padre'"></app-primer-componente>
</section>
```

Una vez este cargado todo el componente podremos disponer de este input que al ser una propiedad de la clase puede renderizarse perfectamente dentro del HTML del propio componente.

Si os acordáis de las funciones de ciclo de vida de los componentes lo input y los outputs se cargan con el ngOnChanges o con el ngOnInit.

```
//OJO DENTRO DEL HTML DEL COMPONENTE NO USAMOS EL
//ALIAS SINO EL NOMBRE DE LA PROPIEDAD
<p>{{texto}}</p>
```

Visualizar el resultado por el navegador, podréis comprobar que os sale el texto que habríais introducido dentro del del input.

Recordar: Ese texto puede ser una variable o constante, así como una cualquiera propiedad que tenga el componente padre.

5.6. Decorador @Output. Del hijo al padre

Este tipo de decorador lo usamos para pasar información del hijo al padre, el padre esperará un aviso o evento por parte del hijo con el que gestionará la información que este le envía.

Para usar un decorador output al igual que el input hay que importarla en cada componente donde se vaya a usar de la siguiente forma.

```
import { Component, EventEmitter, OnInit, Output } from '@angular/core';
```

también debemos importar la librería EventEmitter que es la que me ofrece la posibilidad de emitir un evento personalizado al padre.

Creamos la propiedad que hacer de output y la inicializamos.

```
import { Component, EventEmitter, OnInit, Output } from '@angular/core';
@Component({
  selector: 'app-primer-componente',
  templateUrl: './primer-componentes.component.html',
  styleUrls: ['./primer-componentes.component.css']
})
```

```
export class PrimerComponentesComponent implements OnInit {

  //Creacion de un output
  @Output() delete = new EventEmitter()

  constructor() {
  }

  ngOnInit(): void { }
}
```

Ahora nos toca crear una función que nos permite llenar el output y emitirle el resultado al padre para que este actúe en consecuencia

Por ejemplo, podemos crear en el HTML un evento que nos envíe un número y este se lo comuniquemos al padre, en el evento onDelete vamos a enviar como parámetro un número y este tenemos que hacérselo llegar al padre para que lo saque por consola.

Primer paso crear el evento y recoger el valor que nos envía.

```
<button (click)="onDelete(5)">Enviar numero</button>
```

Este evento se está produciendo en el HTML del componente con lo que tendremos que crear una función onDelete() en el TypeScript de dicho componente de la siguiente manera:

```
import { Component, EventEmitter, OnInit, Output } from '@angular/core';

@Component({
  selector: 'app-primer-componente',
  templateUrl: './primer-componentes.component.html',
  styleUrls: ['./primer-componentes.component.css']
})
```

```
export class PrimerComponentesComponent implements OnInit {


  //Creacion de un output
  @Output() delete = new EventEmitter()

  constructor() {
  }

  ngOnInit(): void { }

  onDelete(numero: number) {
    this.delete.emit(numero)
  }}
```

Al recoger el parámetro número del propio evento este se emite hacia al padre creando un evento personalizado dentro de la etiqueta que carga el propio componente.

En el HTML del componente padre la etiqueta del componente antes mencionado tendrá un evento personalizado que se llamará exactamente igual que el nombre del output declarado. Este evento llamará a una función declarada dentro del padre.

```
<section>
     <app-primer-componente    (delete)="handleDeleteFather($event)"></app-
primer-componente>
</section>
```

Este método o evento personalizado recibe un $event que en este caso representa al valor del número que le estamos pasando por parámetro. Este valor podremos recogerlo a través de la función handleDeleteFather($event) que tendremos que crear dentro del componente padre.

```
import { Component } from '@angular/core';

@Component({
```

```
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {

  handleDeleteFather($event: any) {
    console.log($event)
  }
}
```

Cerrando de esta manera el ciclo en el cual el componente hijo la manda un valor al padre y este es capaz de recogerlo a través de un componente personalizado.

5.7. Referencias bibliográficas

Aristeidis Bampakos, P. D. (2023). *Learning Angular: A no-nonsense guide to building web applications with Angular 15.* Packt Publishing.

Bampakos, A. (2023). *Angular Projects: Build modern web apps in Angular 16 with 10 different projects and cutting-edge technologies.* Packt Publishing.

Google. (11 de 07 de 2024). *Android Developers*. Obtenido de https://developer.android.com/

Microsoft. (15 de 07 de 2024). *TypeScript official web site*. Obtenido de https://www.typescriptlang.org/

Uluca, D. (2024). *Angular for Enterprise Applications - Build scalable Angular apps using the minimalist Router-first architecture.* Packt Publishing.

A fondo

Interpolación {{}} en Angular al detalle

Desarrollo web. (30 de octubre de 2017). *Interpolación {{}} en Angular al detalle* [Blog]. https://desarrolloweb.com/articulos/binding-interpolacion-angular.html

Interesante explicación de cómo se realiza la interpolación de propiedades en Angular.

Input y Output en Angular

Bastidas, W. (2 de agosto de 2020). *Angular decoradores @Input y @Output* [Blog]. https://medium.com/williambastidasblog/angular-decoradores-input-y-output-70af5f43a04

Un blog interesante donde se explican cómo funcionan los componentes en angular.

Property Biding. Angular documentación oficial

Angular. https://angular.io/guide/property-binding

Esta es la documentación oficial de los property biding. Aquí puedes también consultar todas las novedades de las siguientes versiones si es que ocurriera.